

Using Set Theory and Dynamic Data Structures to Manage Linguistic Structures

Cherimekh Samy Firas & Bensefia Abdessamed

ESI ALGIERS

ADDS MODULE

2025-2026

Project Overview

This project tackles the representation of text paragraphs — read from files — as mathematical **sets of unique sentences**, enabling formal set-theoretic operations over natural language data.

Phase 1 — Representation

Parse text files into paragraphs; each paragraph becomes an AVL tree of sentences.

Phase 2 — Set Operations

Apply Union ($\cup$), Intersection ($\cap$), and Difference ($\setminus$) on pairs of paragraph-sets.

Phase 3 — Testing

Validate correctness and performance across multiple files and paragraph combinations.

Why AVL Trees?

Choosing the right data structure is critical. AVL trees outperform linked lists on every metric relevant to set operations over large text corpora.

Operation	Linked List	AVL Tree
Insert	$O(n)$	$O(\log n)$
Search	$O(n)$	$O(\log n)$
Union	$O(n^2)$	$O(n \log n)$
Intersection	$O(n^2)$	$O(n \log n)$
Difference	$O(n^2)$	$O(n \log n)$

No Duplicates

BST property enforces uniqueness — each sentence appears exactly once.

Self-Balancing

Rotations guarantee $O(\log n)$ insert and search — no worst-case degradation.

Search-Heavy Ops

Union, Intersection, Difference all rely on frequent lookups — $O(\log n)$ is critical.

Defined Types & Abstract Machine

AVL Node Structure (C)

```
typedef struct AVLnode {  
    char*    word;  
    int      height;  
    int      balance_f;  
    AVLnode* LC;  
    AVLnode* RC;  
} AVLnode;
```

Each node stores a **sentence string**, its subtree height, balance factor, and pointers to left and right children.

Abstract Machine Interface

Function	Role
<code>create_node(word)</code>	Allocates & initializes a new AVL node
<code>RC(p)</code>	Returns the right child of node p
<code>LC(p)</code>	Returns the left child of node p
<code>Ass_RC(p, N)</code>	Assigns N as the right child of p
<code>Ass_LC(p, N)</code>	Assigns N as the left child of p
<code>height(p)</code>	Returns height of subtree rooted at p
<code>balance_f(p)</code>	Returns $\text{height}(\text{RC}) - \text{height}(\text{LC})$

Key Processing Functions



EndPara(char* string)

Role: Detects the end of a paragraph

Logic: Checks if the last meaningful character of a line is a '.' followed by '\n'. Returns true if the paragraph is complete.



lowerstring(char* str)

Role: Converts to lowercase

Logic: Converts all characters of a string to lowercase using tolower(). Ensures case-insensitive comparison during AVL insertions.



delete_ponct(char* str)

Role: Removes non-letter characters

Logic: Removes all non-letter characters from a string, keeping only a-z, spaces, and dots. Cleans the text before processing.



GetPhrases(char* paragraph)

Role: Splits paragraph into sentences and builds AVL tree

Logic: Splits a cleaned paragraph into individual sentences using '.' as delimiter. Each sentence is trimmed of whitespace and inserted into an AVL tree. Returns the root of the resulting AVL tree representing the paragraph as a set.



GetParagraph(char* FilePath, int* num_paragraphs)

Role: Reads file and processes all paragraphs

Logic: Opens a text file and reads it line by line. Concatenates lines into a paragraph buffer. When EndPara() returns true, it processes the paragraph using lowerstring(), delete_ponct(), and GetPhrases(), then stores the resulting AVL root in a dynamic array. Returns an array of AVL tree roots, one per paragraph.

AVL Rotations — Rebalancing Strategy

After every insertion, balance factors are recomputed via `UpdateHeight_Bf()`. When $|\text{balance_f}| > 1$, one of four rotations restores balance.

Right Rotation (LL)

Left subtree is heavy. Pivot root down-right; left child becomes new root.



Left Rotation (RR)

Right subtree is heavy. Pivot root down-left; right child becomes new root.

Left-Right (LR)

Left child is right-heavy. Apply Left Rotation on left child, then Right Rotation on root.



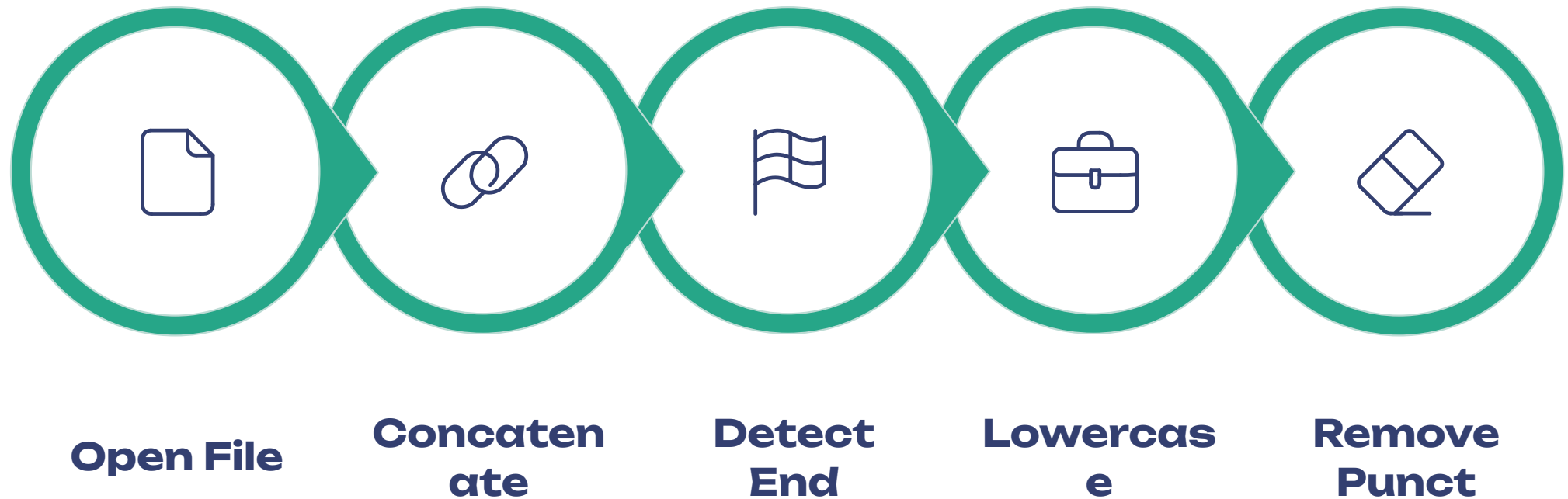
Right-Left (RL)

Right child is left-heavy. Apply Right Rotation on right child, then Left Rotation on root.

 After every rotation, `UpdateHeight_Bf()` is called to refresh heights and balance factors on all affected nodes.

File Processing Pipeline

Each loaded text file is parsed through an 8-stage pipeline. The output is a dynamic array of AVL trees — one per detected paragraph.



The result is a dynamic array where each index corresponds to one paragraph, stored as a balanced AVL tree of its constituent sentences.

Set Operations on Paragraph Trees

All three operations traverse AVL trees in-order and leverage `insert()` and `search()` — both $O(\log n)$ — to build the result tree efficiently.

- ✅ AVL insert automatically ignores duplicates via BST comparison — no explicit deduplication needed.

Union — $A \cup B$

Traverse Tree1 + Tree2 → insert all nodes into result. Duplicates silently ignored by AVL insert. **Result:** all unique sentences from both.

Intersection — $A \cap B$

Traverse Tree1; for each sentence, `search()` in Tree2 ($O(\log n)$). Insert only if found. **Result:** sentences common to both.

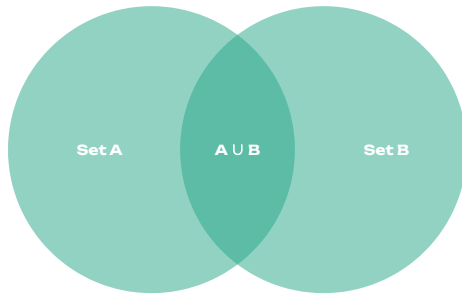
Difference — $A \ominus B$

Traverse Tree1; for each sentence, `search()` in Tree2. Insert only if **not** found. **Result:** sentences exclusive to Tree1.

Set Operations with Venn Diagrams

1

Union ($A \cup B$)

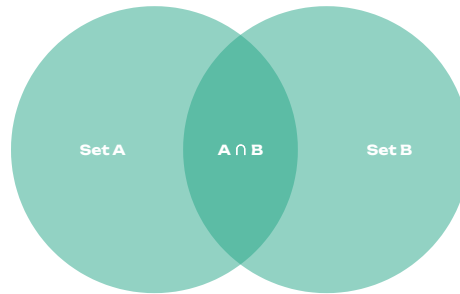


All unique sentences from Paragraph A and Paragraph B combined

Logic: Traverse both trees → insert every node into result (duplicates ignored by AVL)

2

Intersection ($A \cap B$)

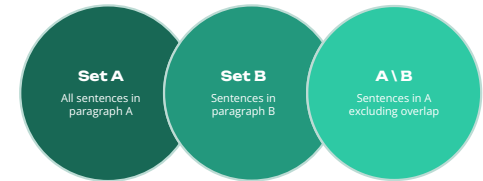


Only sentences that appear in both Paragraph A and Paragraph B

Logic: For each node in Tree1 → search in Tree2 → if found, insert into result

3

Difference ($A \setminus B$)



Sentences in Paragraph A that do not appear in Paragraph B

Logic: For each node in Tree1 → search in Tree2 → if NOT found, insert into result

User Interface & Features

An interactive CLI menu provides full control over files, paragraphs, and set operations.



Load a File

Parse a text file into its paragraphs; each stored as an AVL tree.



Remove a File

Free all allocated memory and update the active file list.



Perform Set Operation

Select Union, Intersection, or Difference on any paragraphs across loaded files. Supports chained operations on N paragraphs.



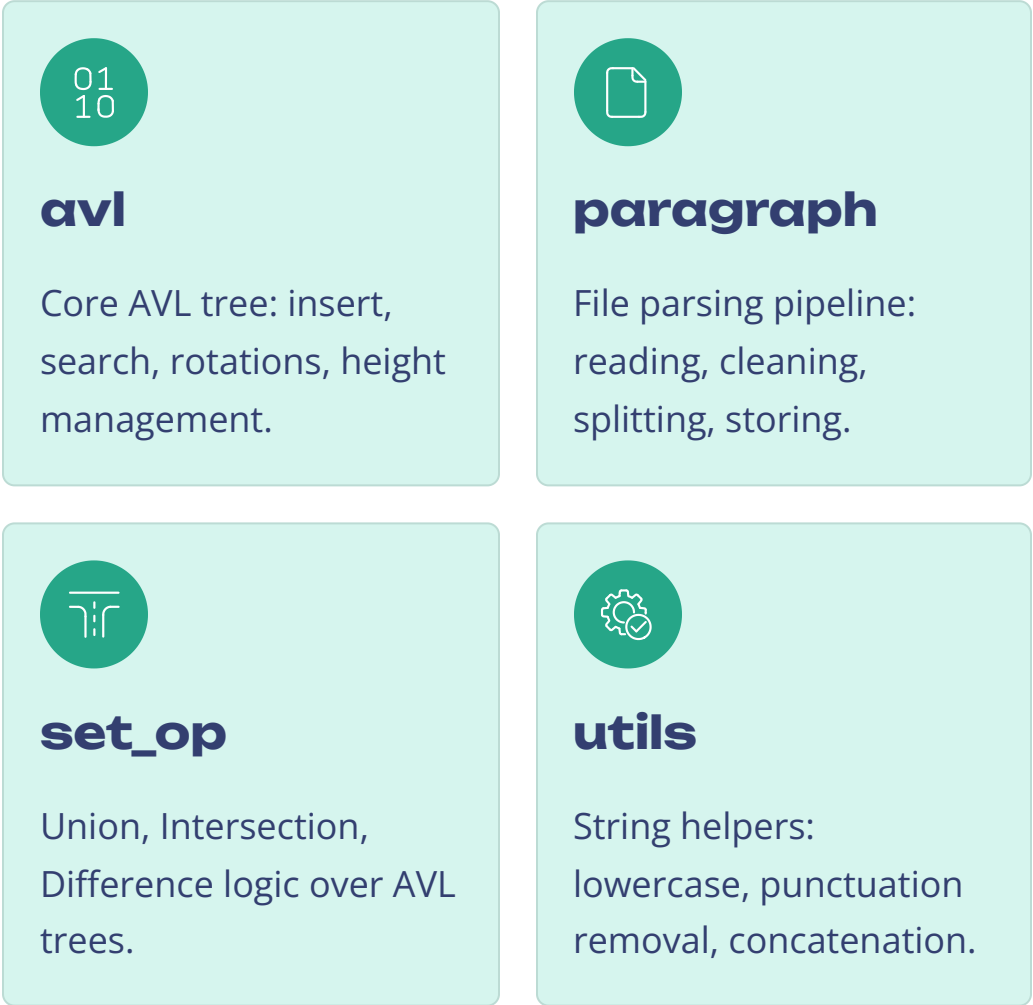
Display Paragraph

Visualize the AVL tree structure of any specific paragraph in-terminal.

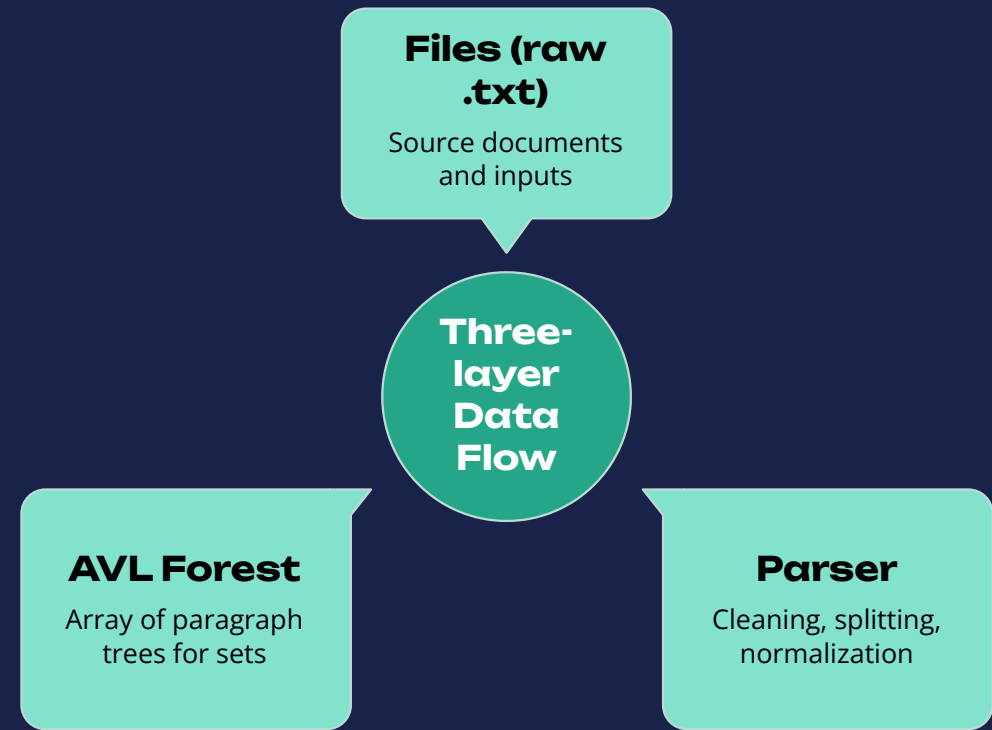
- ✔ Supports up to **10 files simultaneously** with full cross-file set operations.

Architecture Overview

Modular Design



Data Flow



Clean separation of concerns ensures each module is independently testable and maintainable.

Conclusion



Optimal Structure

AVL Trees guarantee uniqueness and $O(\log n)$ performance — the ideal fit for set-based paragraph representation.



Scalable Performance

$O(n \log n)$ set operations scale efficiently to large text corpora across multiple files.



Clean Architecture

The four-module design (avl, paragraph, set_op, utils) enforces strict separation of concerns and simplifies maintenance.



Full Correctness

Union, Intersection, and Difference operate correctly across files, paragraphs, and chained operations.

ESI ALGIERS — ADDS 2025–2026

CHERIMEKH SAMY FIRAS

BENSEFIA ABDESSAMED